AUDIT COMPLET ET RECOMMANDATION DE RECONSTRUCTION

Projet VilleConnect / DourConnect

Analyse de la pré-app Manus et feuille de route vers une version production-grade

Auteur du dossier : JS-Innov.ia

Contexte : reprise d'une pré-app réalisée sur Manus pour réimplantation robuste, scalable et peu sujette aux bugs.

Date de génération : 22/04/2026 14:32

Empreinte SHA256 du dossier : bf92acee1c24ab5ec489629b59bd8b3736b30c7a13eda00c405f33daa1449dea

Résumé exécutif : la base actuelle est pertinente comme prototype et comme preuve de concept, mais elle n'est pas encore suffisamment industrialisée pour supporter un déploiement sérieux, un branding local dynamique par code postal, un moteur promotions/courses, ni un module carburant fiable. La bonne stratégie n'est pas une simple migration d'hébergement : c'est une réimplémentation structurée en conservant la vision produit, certains écrans, et la logique UX locale.

1. Ce qui a été analysé

Deux archives ont été partagées. La seconde contient la pré-app principale. L'analyse a porté sur la structure projet, la stack, les écrans existants, la couche backend, l'authentification, la base de données, les dépendances et la capacité du projet à évoluer vers une version premium sans bugs majeurs.

- Archive légère de démonstration avec documents : design, TODO, PDF de route et quelques écrans.
- Archive principale Expo / React Native avec routing, composants, serveur Node, tRPC, Drizzle et données mockées.
- Fichiers particulièrement révélateurs : app.config.ts, constants/oauth.ts, hooks/use-auth.ts, lib/_core/auth.ts, server/README.md, drizzle/schema.ts, server/routers.ts, data/mock-data.ts, design.md.

Constat global : le projet est aujourd'hui plus avancé sur la couche démonstration UX que sur la couche métier, données et robustesse runtime.

2. Ce qui est déjà bon dans la pré-app

Le prototype n'est pas à jeter. Il contient plusieurs éléments stratégiques qu'il faut conserver.

- **Vision produit cohérente** : application citoyenne locale mêlant commerces, commune, garages/artisans, démarches citoyennes et profil.
- **Base mobile moderne** : Expo 54, React Native, Expo Router, React Query et tRPC donnent une base de développement rapide.
- **UX lisible** : les écrans Accueil, Commerces, Commune, DCI et Profil structurent déjà un parcours utilisateur clair.
- **Design document propre** : palette, onglets, flux principaux et composants réutilisables sont suffisamment définis pour servir de base au redesign production.
- **Intuition produit forte** : la logique "ville connectée" locale est bonne et extensible à DourConnect puis à d'autres villes.

Conclusion de cette partie : l'idée, l'expérience cible et plusieurs briques visuelles méritent d'être conservées. Ce qui doit changer, c'est l'architecture, pas l'ambition.

3. Faiblesses majeures détectées

Les risques ci-dessous expliquent pourquoi la version actuelle ne doit pas être poussée en production telle quelle.

- **Dépendance Manus trop forte** : bundle ID de type space.manus..., schéma deep-link dérivé Manus, OAuth Manus, runtime Manus et conventions d'environnement très spécifiques.
- **Backend métier presque vide** : la structure serveur existe, mais le coeur fonctionnel n'est pas réellement implémenté.
- **Base de données sous-dimensionnée** : le schéma est très pauvre par rapport aux ambitions produit.
- **Usage massif de données mockées** : commerces, garages, alertes et demandes sont principalement simulés côté front.
- **Authentification fragile en multi-plateforme** : mélange cookie web, token natif, cache local, redirections OAuth et logique dépendante de l'environnement.
- **Pas de vraie stratégie performance/fiabilité** : peu d'indications sur monitoring, observabilité, retry, journaux, tests de charge, feature flags, analytics produit, résilience réseau.

Zone	Etat actuel	Risque
Auth	Hybride et spécifique Manus	Déconnexions, comportements divergents web/mobile
Données	Faible schéma + mocks	Impossible de scaler proprement

Métier	Routes minces	UX trompeuse car peu connectée au réel
Déploiement	Bundle et OAuth Manus	Portabilité limitée
Notifications	Non industrialisées	Promesses produit non tenables

Décision recommandée : ne pas "porter" l'application telle quelle vers un autre hébergement. Il faut la replanifier comme un produit indépendant, avec séparation propre entre front, back-office, moteur de données et agents intelligents.

4. Lecture produit : ce que VilleConnect doit devenir

La conversation a progressivement clarifié une ambition bien plus forte qu'une simple app locale. Le coeur de différenciation n'est pas seulement la présence de commerces ou d'alertes communales. Le vrai levier, c'est la personnalisation par territoire et l'orchestration intelligente d'économies utiles.

- **Nom dynamique par code postal** : ex. 7370 -> DourConnect. L'utilisateur a l'impression d'avoir l'application de sa ville.
- **VilleConnect OS en coeur de plateforme** : moteur commun multi-villes, multi-modules.
- **Module courses / économies** : SmartCart / EcoPanier AI pour les promotions, paniers et menus.
- **Module carburant** : notifications avec les 3 pompes les moins chères autour de l'utilisateur.
- **Module citoyen** : DCI, commune, rendez-vous, signalements, informations utiles.
- **Module commerces locaux** : vitrines, promotions, visibilité, prise de contact.

Autrement dit : DourConnect est l'instance locale visible. VilleConnect OS est le moteur. Cette séparation permet le branding local sans casser la scalabilité.

5. Recommandation technique cible

Pour obtenir une version super performante et robuste, voici l'architecture recommandée.

- **App mobile citoyen** : React Native + Expo maintenu, mais nettoyé des dépendances Manus.
- **Back-office web** : Next.js pour gérer villes, contenus, commerces, promotions, stations-service, demandes citoyennes et dashboards.
- **API backend** : NestJS ou Fastify structuré. Objectif : séparation nette des modules métier, sécurité et tests.
- **Base de données** : PostgreSQL recommandé à la place de MySQL pour robustesse, extension et géodonnées plus simples à exploiter.
- **ORM** : Prisma ou Drizzle, avec schéma métier complet et migrations propres.
- **Notifications** : Expo Notifications / OneSignal + logique serveur d'orchestration.
- **Cartographie** : Mapbox ou Google Maps selon budget.
- **Jobs et agents** : workers planifiés pour promos, carburant, notifications et classement.

Cette architecture garde la vélocité du mobile moderne, mais introduit la discipline nécessaire pour un produit réel : modularité, traçabilité, monitoring et capacité à ajouter des agents IA sans transformer le projet en monolithe fragile.

6. Nouveau modèle de données minimum

Le schéma actuel ne peut pas porter l'ambition du produit. Voici le socle minimal conseillé.

Domaine	Tables / entités minimales
Territoire	cities, postal_codes, neighborhoods, city_settings
Utilisateurs	users, user_profiles, user_preferences, user_devices, user_notification_settings
Commerces	shops, shop_categories, shop_offers, shop_media, shop_opening_hours, shop_contacts

Garages / artisans	garages, garage_services, quotes, appointments
Citoyen	civic_requests, civic_offers, civic_messages, civic_status_history
Commune	communal_alerts, communal_services, municipal_pages, municipal_appointments
Carburant	fuel_stations, fuel_prices, fuel_price_history, user_vehicles, fuel_alerts
Courses	products, retailers, retailer_prices, shopping_lists, meal_plans, pantry_items
Technique	audit_logs, jobs, notification_events, feature_flags

Le principe à retenir : chaque promesse produit doit avoir sa représentation en base, son historique et sa traçabilité. Sans cela, les fonctionnalités paraissent belles en démo mais cassent dès qu'on veut personnaliser, notifier ou analyser.

7. Modules MVP recommandés

Le MVP ne doit pas tout faire. Il doit prouver la valeur locale, la rétention et la capacité du moteur à se décliner par ville.

- **MVP 1 - Onboarding géographique** : choix ou détection du code postal, branding dynamique DourConnect / autre ville.
- **MVP 2 - Accueil intelligent** : carte, blocs locaux, promotions en vedette, alertes utiles, bouton carburant.
- **MVP 3 - Commerces** : annuaire local réel, catégories, fiches détaillées, promotions, favoris.
- **MVP 4 - Commune** : infos utiles, contacts, alertes, liens administratifs, rendez-vous si possible.
- **MVP 5 - DCI** : expression libre du besoin, statut, réception d'offres, historique.
- **MVP 6 - FuelSaver** : top 3 des stations-service les moins chères, économie estimée, distance, notification contextuelle.

Les modules courses / EcoPanier AI peuvent arriver juste après le MVP si les données promotions sont réellement exploitables. Ils ont un fort potentiel, mais nécessitent plus de collecte et de normalisation de données.

8. Module carburant : innovation à garder absolument

L'ajout le plus différenciant de la conversation est le module carburant. Il améliore énormément la valeur perçue.

- Comparer en temps quasi réel les prix carburant par station autour de l'utilisateur.
- Pousser une notification locale avec les 3 pompes les moins chères de la ville ou du trajet habituel.
- Afficher l'économie estimée par plein.
- Prendre en compte le type de carburant et, à terme, le véhicule de l'utilisateur.
- Rendre l'alerte actionnable : distance, ETA, ouverture de la navigation.

Exemple d'alerte : « DourConnect - Les 3 stations les moins chères autour de toi : Q8 1,61 EUR/L, Total 1,63 EUR/L, Esso 1,64 EUR/L. Économie estimée : 8,70 EUR sur ton plein. »

Ce module a trois avantages stratégiques : utilité concrète, récurrence quotidienne et excellente compatibilité avec le branding local.

9. Pourquoi il faut réimplémenter plutôt que corriger

- La dépendance environnementale au runtime Manus coûtera plus cher à retirer au fil de l'eau qu'à replanifier proprement.
- Le schéma de données actuel est trop en retrait par rapport au produit visé.
- Le mix auth web/natif/cache local est à haut risque pour les bugs fantômes.

- La présence de beaucoup de mocks donne une illusion d'avancement qui ralentit souvent les vraies décisions techniques.
- Le projet va recevoir des modules additionnels (branding local, carburant, menus, promotions, notifications intelligentes) ; il faut une base saine maintenant.

Conserver : vision, wording, hiérarchie de certains écrans, tonalité locale, éléments design. Reprendre à neuf : auth, API, données, jobs, notifications, logique métier et architecture de déploiement.

10. Plan de migration recommandé

Phase	Objectif	Livrables
1. Cadrage	Figer le produit cible	Modules MVP, branding, personas, règles villes
2. Modélisation	Refaire les données	Schéma PostgreSQL, rôles, permissions, contrats API
3. Backend	Créer le socle fiable	Auth, users, cities, shops, civic, fuel, notifications
4. Front mobile	Refondre les flows essentiels	Onboarding, home, shops, commune, DCI, fuel
5. Back-office	Rendre le système pilotable	CRUD villes, commerces, promos, alertes, stations
6. Intelligence	Ajouter le moteur d'optimisation	Classement carburant, promos, scoring, suggestions
7. Industrialisation	Réduire les bugs	Tests, monitoring, analytics, gestion incidents, stores

Important : ne pas essayer d'implémenter toutes les promesses de la conversation dans le premier sprint. La vraie performance vient d'un périmètre serré, mesurable et bien testé.

11. Priorités anti-bug et performance

- Authentification unifiée et documentée.
- Gestion stricte des erreurs API côté mobile.
- Validation serveur de toutes les écritures.
- Historique et logs des notifications envoyées.
- Pagination sur les listes et cache intelligent côté mobile.
- Monitoring des crashes et erreurs réseau.
- Feature flags pour déployer les modules progressivement.
- Tests d'intégration sur les flux critiques : onboarding, connexion, consultation commerce, création DCI, réception notification carburant.
- Back-office pour corriger les contenus sans redéploiement mobile.

Une application "sans bug" n'existe pas, mais une application conçue pour limiter, détecter et corriger rapidement les bugs existe. C'est cette philosophie qu'il faut adopter pour DourConnect.

12. Positionnement final conseillé

Le bon positionnement n'est pas « simple annuaire communal ». Le bon positionnement est : plateforme locale intelligente qui aide le citoyen à gagner du temps, de l'argent et de la clarté dans sa ville.

- **Nom visible** : DourConnect, MonsConnect, etc.
- **Moteur interne** : VilleConnect OS.
- **Promesse utilisateur** : informations locales utiles + économies actionnables.
- **Différenciation** : hyper-local, notifications intelligentes, multi-modules, branding géographique automatique.

Cette stratégie donne une vraie cohérence produit : on ne vend pas une app générique, on déploie un réseau d'instances locales sur un moteur commun.

13. Décision finale recommandée

Décision recommandée : engager une réimplémentation premium à partir du concept et des meilleurs écrans de la pré-app Manus, avec priorité à DourConnect comme pilote local.

- Ne pas migrer la pré-app telle quelle.
- Conserver la vision et certains écrans comme référence UX.
- Rebâtir le coeur technique indépendamment de Manus.
- Lancer un MVP propre sur une ville pilote.
- Ajouter très tôt le module carburant comme levier différenciant.

Verdict synthétique : la pré-app actuelle est une excellente maquette stratégique, pas encore un produit de production. Le projet mérite d'être reconstruit sérieusement, car l'idée est plus forte que l'implémentation actuelle.

14. Notice de paternité et protection

Titulaire : Julien Pagin - Entité : JS-Innov.ia. Le présent dossier documente une analyse, un concept produit, une architecture recommandée et une feuille de route de réimplémentation. Il sert de preuve de cadrage et de dissuasion intellectuelle. Il ne constitue pas une garantie d'issue contentieuse. Pour une protection renforcée, prévoir dépôt i-DEPOT, preuve d'horodatage ou archivage qualifié si nécessaire.